EXPERIMENT: A* ALGORITHM

Aim

To write a Python program to implement the A* search algorithm.

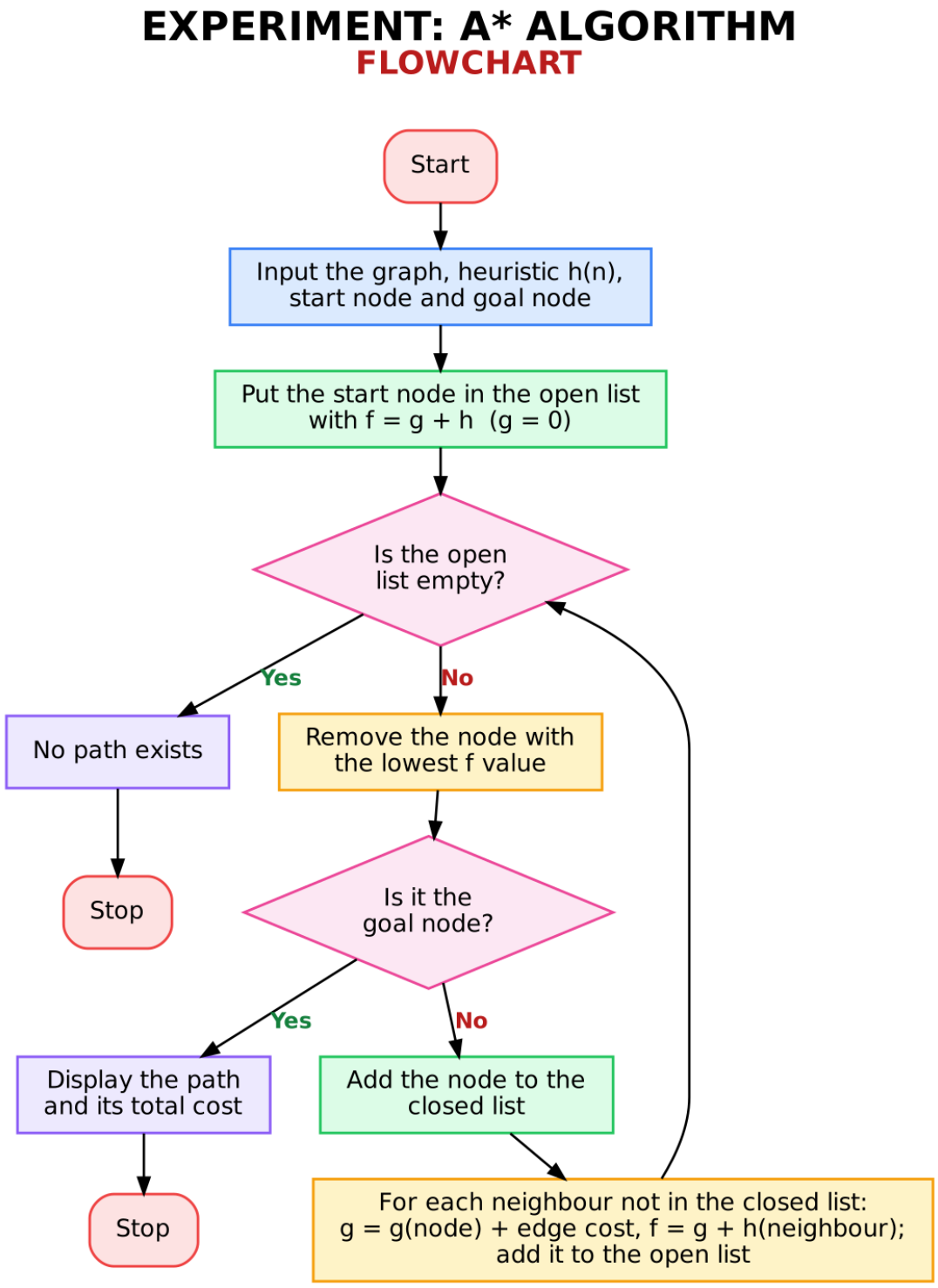
Objective

- To understand informed (heuristic) search techniques.
- To implement A* using the evaluation function f(n) = g(n) + h(n) with a priority queue.
- To find the least-cost path from the start node to the goal node.

Algorithm

1. Start.
2. Define the weighted graph, the heuristic value h(n) of every node, the start node and the goal node.
3. Insert the start node into the open list (priority queue) with f = g + h, where g = 0.
4. Remove the node with the lowest f value from the open list.
5. If it is the goal node, display the path and the total cost, then stop.
6. Add the node to the closed list.
7. For every neighbour that is not in the closed list, calculate g = g(node) + edge cost and f = g + h, and insert it into the open list.
8. Repeat steps 4 to 7 until the goal is found or the open list becomes empty.
9. If the open list is empty, display that no path exists. Stop.

Flowchart:



Code (Python)

```
import heapq

def a_star(graph, heuristic, start, goal):
    # priority queue holds (f, g, node, path)
    open_list = [(heuristic[start], 0, start, [start])]
    closed = set()
```

```
    while open_list:
        f, g, node, path = heapq.heappop(open_list)

        if node == goal:
            return path, g

        if node in closed:
            continue
        closed.add(node)

        for neighbour, cost in graph[node]:
            if neighbour not in closed:
                new_g = g + cost
                new_f = new_g + heuristic[neighbour]
                heapq.heappush(open_list, (new_f, new_g, neighbour, path + [neighbour]))

    return None, None

# Weighted graph: node -> [(neighbour, edge cost)]
graph = {
    'S': [('A', 1), ('B', 4)],
    'A': [('B', 2), ('C', 5), ('D', 12)],
    'B': [('C', 2)],
    'C': [('D', 3), ('G', 7)],
    'D': [('G', 2)],
    'G': []
}

# Heuristic: estimated cost from each node to the goal G
heuristic = {'S': 7, 'A': 6, 'B': 4, 'C': 4, 'D': 2, 'G': 0}

path, cost = a_star(graph, heuristic, 'S', 'G')

print("Path found by A*:", " -> ".join(path))
print("Total cost      :", cost)
```

Output

Python Shell - Output

```
Path found by A*: S -> A -> B -> C -> D -> G
Total cost      : 10
```

Output Result

Thus, the Python program to implement the A* algorithm was executed successfully. The least-cost path S -> A -> B -> C -> D -> G was found with a total cost of 10.

Conclusion

The experiment helped in understanding informed search. A* combines the cost already travelled g(n) with the estimated remaining cost h(n), and it returns the optimal path when the heuristic never overestimates the true cost.